

RÉPUBLIQUE DU CAMEROUN

Paix - Travail - Patrie

UNIVERSITÉ DE YAOUNDÉ I

FACULTÉ DES SCIENCES

DÉPARTEMENT D'INFORMATIQUE



REPUBLIC OF CAMEROON

Peace - Work - Fatherland

UNIVERSITY OF YAOUNDE I

FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER
SCIENCE

INF 4258 : Projet

Simulateur de gestion de la sécurité dans les réseaux véhiculaires ad hoc (VANETs)

*Architecture Cyber-Physique, Contrôle IA (PPO)
et Évaluation face aux Attaques Adversariales (cGAN)*

Rédigé et présenté par :

DASSI MANDJO LEA JUSTINE	22W2164
MELONG LETHYCIA	22W2147
NGUEFACK TANGOMO CHRIS ARTHUR	25G2055

Projet disponible sur :

<https://github.com/Tower0X/sumo-rl-gan>

Sous l'encadrement de :

Pr. Paulin MELATAGIA

Enseignant à l'Université de Yaoundé I

Année académique : 2025-2026

Résumé

Les réseaux de véhicules ad hoc (VANETs) constituent le pilier technologique des Systèmes de Transport Intelligents (ITS). Ils permettent aux véhicules de communiquer entre eux (V2V) et avec l'infrastructure routière (V2I), ouvrant la voie à une gestion autonome et efficace du trafic urbain. Cependant, la quasi-totalité des travaux existants sur le contrôle de trafic par intelligence artificielle ignore la dimension réseau : les agents sont entraînés dans des environnements parfaits où les données arrivent instantanément, sans bruit ni cyberattaques.

Ce rapport présente la conception et l'implémentation d'un simulateur VANET cyber-physique réaliste dans lequel un agent d'Apprentissage par Renforcement Profond (PPO) est formé à adopter un comportement dit « Fail-Safe » – garantissant zéro collision même sous l'action d'un attaquant neuronal génératif (cGAN). Le système modélise mathématiquement la gigue réseau (jitter gaussien), les pannes d'antenne RSU, et trois familles d'attaques (Temporal DoS, Data Poisoning, Ghost Vehicles / Attaque Sybil). L'architecture passe à l'échelle grâce à un cadre Multi-Agent Reinforcement Learning (MARL) avec partage de paramètres (*Parameter Sharing*) sur une topologie urbaine 4×4.

Les résultats expérimentaux démontrent : (i) une convergence stable de l'agent PPO en moins de 20 000 itérations, (ii) une résilience totale (0 collision physique) même sous les attaques combinées DoS + Data Poisoning, et (iii) une propriété d'immunité collective instantanée conférée par le partage de paramètres.

Mots-clés : VANET, Deep Reinforcement Learning, PPO, cGAN, cybersécurité, Fail-Safe, SUMO, MARL, TraCI.

Table des matières

Résumé	i
1 Introduction	1
2 Revue des solutions existantes	2
2.1 Définition des concepts fondamentaux	2
2.1.1 Réseaux VANET	2
2.1.2 Apprentissage par Renforcement (RL)	2
2.1.3 Proximal Policy Optimization (PPO)	2
2.1.4 Réseaux Antagonistes Génératifs (GAN)	2
2.2 Solutions existantes	3
2.2.1 Contrôle de trafic par RL : sumo-rl	3
2.2.2 MARL pour le contrôle multi-intersections	3
2.2.3 Sécurité des VANETs	3
2.3 Positionnement	3
3 Méthodologie et approche	4
3.1 Présentation des données	4
3.2 Méthodes choisies	4
3.2.1 Architecture hybride Physics-AI	4
3.2.2 Transformation de la base sumo-rl	4
3.2.3 Choix de PPO	4
3.2.4 Récompense Fail-Safe	5
3.2.5 Modélisation des cyberattaques	5
3.2.6 L'Attaquant Neuronal (cGAN) et le Surrogate Model	6
3.2.7 Architecture MARL : concept et implémentation	6
3.3 Métriques d'évaluation	7
4 Implémentation et résultats	8
4.1 Architecture de l'application	8
4.2 Environnement de développement	8
4.2.1 Choix de SUMO contre Veins/OMNeT++	8
4.3 Résultats obtenus	8
4.3.1 Entraînement de l'agent PPO défensif	8
4.3.2 Entraînement de l'attaquant cGAN	9
4.3.3 Confrontation PPO vs. cGAN – Scénarios expérimentaux	9
4.4 Interprétation des résultats	10
4.5 Analyse critique	10
4.6 Code source	11

4.7	Limites de l'approche	11
4.8	Apports du projet	11
5	Considérations éthiques	12
5.1	Protection des données personnelles	12
5.2	Biais des données et discrimination algorithmique	12
5.3	Transparence et explicabilité	12
5.4	Responsabilité et usage	13
5.5	Consentement et finalité	13
6	Conclusion générale	14
6.1	Rappel du problème et récapitulatif des objectifs atteints	14
6.2	Bilan global	14
6.3	Ouvertures possibles	14
	Bibliographie	15

1. Introduction

L'urbanisation croissante accentue la congestion routière, avec des coûts économiques, environnementaux et sécuritaires importants. Les Systèmes de Transport Intelligents, et en particulier les réseaux VANET, promettent une coordination autonome des véhicules, mais leur topologie hautement dynamique impose des contraintes de latence critiques (moins de 100 ms pour les messages vitaux, norme IEEE 802.11p).

La recherche actuelle en contrôle de trafic par IA souffre d'un biais majeur : elle suppose une connaissance parfaite et instantanée de l'état du réseau, ignorant les aléas réels des télécommunications (interférences, pertes de paquets) ainsi que les cyberattaques délibérées (déné de service, attaques Sybil, empoisonnement de données). Ce projet, mené dans le cadre de l'UE Projet INF4258 sous la supervision du Pr. Paulin Melatagia, vise donc à concevoir un simulateur VANET réaliste intégrant ces défaillances, et à y entraîner un agent de contrôle de trafic résilient face à un attaquant neuronal autonome.

Cette problématique dépasse l'optimisation académique : un agent dupé par de fausses données peut provoquer des accidents, un risque d'autant plus pressant que les villes intelligentes, y compris au Cameroun, ne pourront adopter ces technologies que si elles sont robustes aux pannes et au sabotage. Les bénéficiaires potentiels incluent les autorités municipales, les constructeurs automobiles, les équipes de cybersécurité et les chercheurs en ITS/RL.

Les objectifs se déclinent ainsi : développer un environnement de simulation VANET network-aware ; entraîner un agent PPO à récompense Fail-Safe garantissant l'absence de collision ; implémenter un arsenal de cyberattaques via un intergiciel Man-In-The-Middle ; concevoir un attaquant neuronal (cGAN) appuyé sur un modèle Surrogate différentiable pour contourner la non-dérivabilité de SUMO ; et démontrer la scalabilité via une architecture MARL à Parameter Sharing sur une grille 4×4.

La méthodologie s'appuie sur la base open-source sumo-rl, transformée pour intégrer des couches cybernétiques. L'agent de défense est entraîné dans cet environnement bruité, puis confronté à l'attaquant adversarial dans une arène min-max, avant validation sur une topologie multi-intersections.

Le rapport s'organise comme suit : le Chapitre 2 présente l'état de l'art, le Chapitre 3 la méthodologie, le Chapitre 4 l'implémentation et les résultats, le Chapitre 5 les considérations éthiques, et le Chapitre 6 conclut avec des perspectives.

2. Revue des solutions existantes

2.1 Définition des concepts fondamentaux

2.1.1 Réseaux VANET

Un réseau VANET est un réseau ad hoc mobile dans lequel chaque nœud est un véhicule. Les communications s'effectuent selon deux modes : Vehicle-to-Vehicle (V2V) pour la coopération directe entre véhicules, et Vehicle-to-Infrastructure (V2I) pour l'échange avec les unités de bord de route (Road Side Units, RSU). La norme IEEE 802.11p (DSRC) et son évolution C-V2X (Cellular V2X) constituent les standards radio utilisés.

Les caractéristiques distinctives des VANETs sont : une topologie ultra-dynamique, une densité variable de nœuds, des contraintes de latence strictes (< 100 ms pour les messages de sécurité), et une exposition aux conditions radio urbaines (multi-trajet, atténuation, interférences).

2.1.2 Apprentissage par Renforcement (RL)

L'Apprentissage par Renforcement formalise un problème de décision séquentielle par un Processus de Décision Markovien (MDP) (S, A, P, R, γ) , où S est l'espace d'états, A l'espace d'actions, P la fonction de transition, R la récompense, et $\gamma \in [0, 1]$ le facteur d'actualisation. Un agent apprend une politique $\pi : S \rightarrow A$ maximisant la somme des récompenses futures actualisées $\sum_{t=0}^{\infty} \gamma^t R_t$.

2.1.3 Proximal Policy Optimization (PPO)

PPO est un algorithme Policy Gradient développé par OpenAI (Schulman et al., 2017). Sa caractéristique fondamentale est sa fonction objectif écrêtée :

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.1)$$

où $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ est le ratio de politique et $\hat{A}_t$ l'estimateur d'avantage. L'hyperparamètre $\epsilon = 0.2$ empêche les mises à jour trop agressives, conférant une convergence monotone particulièrement adaptée aux environnements stochastiques.

2.1.4 Réseaux Antagonistes Génératifs (GAN)

Un GAN (Goodfellow et al., 2014) met en compétition un Générateur G et un Discriminateur D dans un jeu min-max :

$$\min_G \max_D \mathbb{E}_x [\log D(x)] + \mathbb{E}_z [\log(1 - D(G(z)))] \quad (2.2)$$

Le Générateur apprend à produire des données indiscernables des données réelles, tandis que le Discriminateur apprend à distinguer réel et synthétique.

2.2 Solutions existantes

2.2.1 Contrôle de trafic par RL : sumo-rl

Le framework `sumo-rl` (Lucas Alegre, 2019) est la référence open-source pour le contrôle de feux par RL. Il fournit un encapsuleur (*wrapper*) entre le simulateur SUMO et les standards Gymnasium/PettingZoo. **Forces** : interface standardisée, compatibilité avec l'ensemble des algorithmes SB3, support multi-agents natif. **Faiblesses** : environnement déterministe (latence nulle), récompense purement cinématique sans contrainte de sécurité, algorithmes tabulaires obsolètes.

2.2.2 MARL pour le contrôle multi-intersections

Des travaux comme RESCO (Ault et Wade, 2021) ont appliqué plusieurs algorithmes MARL (MPLight, CoLight) sur des benchmarks urbains réalistes. Ces approches montrent des gains de 20 à 40 % sur le temps d'attente cumulé. Toutefois, elles opèrent sous l'hypothèse d'un réseau parfait et n'adressent pas la résilience aux attaques.

2.2.3 Sécurité des VANETs

La littérature sur la sécurité VANET identifie plusieurs familles d'attaques : les attaques Sybil (faux identifiants), les attaques DoS/DDoS sur la couche MAC, et le Data Poisoning des messages V2X. Les défenses proposées reposent majoritairement sur des mécanismes cryptographiques ou de certification, sans intégrer l'IA comme vecteur de résilience.

2.3 Positionnement

Notre contribution se situe à l'intersection de trois domaines : la simulation VANET réaliste, la cybersécurité des ITS, et le Deep Reinforcement Learning. Contrairement aux travaux existants, nous proposons une **architecture unifiée** qui (1) modélise mathématiquement les dégradations réseau dans l'espace d'état de l'agent, (2) redéfinit la récompense autour de la sécurité physique plutôt que la fluidité seule, et (3) inclut un adversaire neuronal capable de découvrir des vulnérabilités spatio-temporelles autonomement.

3. Méthodologie et approche

3.1 Présentation des données

Le projet repose sur un environnement de simulation dynamique plutôt qu'un jeu de données statique. SUMO génère à chaque pas de temps ($\Delta t = 0.1$ s) la télémétrie de chaque véhicule (position, vitesse, accélération, voie, attente), accessible via l'API TraCI. Le débit d'arrivée suit une loi de Weibull, avec routes pré-générées par `randomTrips.py` et initialisation en régime permanent pour réduire le warmup.

L'observation brute est enrichie par notre couche réseau avant d'être présentée à l'agent :

$$S_t = \left(\text{PhaseID}_{1\dots N}, \text{MinGreen}, \text{Densité}_{1\dots M}, \text{Queue}_{1\dots M}, L_t^{\text{norm}}, C_{\text{ok}} \right)^T \quad (3.1)$$

où L_t^{norm} est la latence normalisée et $C_{\text{ok}} \in \{0, 1\}$ la santé de l'antenne RSU. La latence est modélisée par un jitter gaussien :

$$L_t = \max\left(0, \mu_{\text{base}} + \mathcal{N}(0, \sigma^2)\right) \quad (3.2)$$

avec $\mu_{\text{base}} = 10$ ms, $\sigma = 5$ ms, forçant l'agent à une robustesse structurelle au bruit.

3.2 Méthodes choisies

3.2.1 Architecture hybride Physics-AI

Trois couches : **SUMO** (moteur physique, modèle de Krauss), **TraCI** (pont TCP/IP à 10 Hz) et **VanetGymEnv** (encapsuleur `gymnasium.Env`).

3.2.2 Transformation de la base sumo-rl

La base `sumo-rl` présentait trois lacunes : absence de modélisation réseau, récompense dangereuse ignorant la physique de freinage, et algorithmes tabulaires non scalables. Nous l'avons refondue : suppression des algorithmes obsolètes, injection des couches OSI dans l'observation, refonte de la récompense.

3.2.3 Choix de PPO

Face à l'instabilité des environnements stochastiques, PPO garantit une évolution conservatrice de la politique via son clipping (mise à jour bornée à $\pm 20\%$), contrairement au DQN sujet à divergence :

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3.3)$$

3.2.4 Récompense Fail-Safe

Deux phases sélectionnées par C_{ok} :

Mode nominal ($C_{ok} = 1$) :

$$R_{cin} = \sum_v \begin{cases} -5.0 & a_v < -4.5 \text{ m/s}^2 \\ -1.0 & -4.5 \leq a_v < -2.5 \text{ m/s}^2 \\ 0 & \text{sinon} \end{cases} \quad (3.4)$$

Une décélération inférieure à -4.5 m/s^2 correspond à un freinage d'urgence. La pénalité de -5.0 ($50\times$ la pénalité de file d'attente normale) force l'agent à développer de la souplesse et de l'anticipation des flux inertiels.

Mode dégradé ($C_{ok} = 0$) :

$$R_{fail_safe} = - \sum_i (Q_i \times \lambda_{urgence})^2 \quad (3.5)$$

La pénalité quadratique fait émerger spontanément un cycle de feux « horloge suisse » comportement Fail-Safe.

3.2.5 Modélisation des cyberattaques

Une cyberattaque est définie comme une altération délibérée et adaptative du canal V2I, caractérisée par (Type, Cible, Intensité $I \in [0, 1]$), via l'orchestrateur Man-In-The-Middle :

$$\tilde{S}_{corrompu} = f_{malveillante}(S_{vrai}, \text{VecteurAttaque}) \quad (3.6)$$

1. Temporal DoS (Déni de Service Temporel). Cette attaque cible la couche MAC/Réseau, simulant un brouillage radio ou une saturation du canal CSMA/CA.

Mécanisme : la latence normalisée L_t^{norm} est écrasée par un délai massif (jusqu'à 10s, contre 10-15 ms en nominal) ; si $I > 0.8$, le flag $C_{ok} = 0$ est forcé, simulant une coupure totale du RSU.

Impact : privé de données fraîches, l'agent PPO bascule en mode aveugle – sans le mécanisme Fail-Safe, il opérerait sur des données périmées, menant à un crash du système.

2. Data Poisoning (Empoisonnement des Données). Cette attaque cible la couche Application et masque la gravité réelle de la congestion.

Mécanisme : le vecteur des files d'attente Q est atténué :

$$\tilde{Q} = Q_{\text{réel}} \times (1 - I) \quad (3.7)$$

Impact : avec $I = 1.0$, l'agent reçoit $\tilde{Q} = 0$ et ne perçoit aucune urgence à accorder le feu vert, paralysant l'axe concerné.

3. Ghost Vehicles (Attaque Sybil). À l'inverse du Data Poisoning, l'attaque Sybil crée de la congestion imaginaire en injectant de faux identifiants de véhicules.

Mécanisme : le vecteur de densité est saturé artificiellement :

$$\tilde{D} = \min(1.0, D_{\text{réel}} + I) \quad (3.8)$$

Impact : l'agent accorde de longues phases vertes à des axes vides, au détriment des axes réellement congestionnés.

Justification de la limitation à trois familles d'attaques. Plutôt que de couvrir l'ensemble du spectre des menaces VANET (usurpation cryptographique, relais GPS, jamming multi-bande, attaques blackhole), ce choix répond à deux impératifs : ces trois attaques couvrent orthogonalement Disponibilité (DoS), Intégrité (Data Poisoning) et Authenticité (Sybil) sans redondance ; et elles s'expriment comme de simples transformations algébriques de S_t , compatibles avec le Surrogate Model différentiable – contrairement au jamming multi-bande, qui exigerait une modélisation PHY hors du cadre OSI simplifié retenu pour préserver la vitesse de simulation. C'est donc un compromis entre richesse du modèle de menace et faisabilité computationnelle.

3.2.6 L'Attaquant Neuronal (cGAN) et le Surrogate Model

SUMO étant un moteur discret non différentiable, un **Surrogate Model** contourne ce verrou.

Discriminateur D : LSTM (128 unités) plutôt qu'un MLP classique, pour capturer la dépendance temporelle du dommage (accumulation de file, persistance d'une coupure). Entraîné par régression MSE sur une fenêtre glissante (État, Attaque $\rightarrow$ Dégât) :

$$\mathcal{L}_D = \mathbb{E}_{S,A} \left[(D(S, A) - R_{\text{réel}})^2 \right] \quad (3.9)$$

D , étant un réseau PyTorch, possède un gradient ∇D exploitable.

Générateur G : produit $A_{\text{gen}} = G(S)$ maximisant le dommage prédit :

$$\mathcal{L}_G = -\mathbb{E}_S [D(S, G(S))] \quad (3.10)$$

3.2.7 Architecture MARL : concept et implémentation

Qu'est-ce qu'une architecture MARL ? Le *Multi-Agent Reinforcement Learning* (MARL) étend le RL classique en faisant interagir **plusieurs agents** au sein d'un envi-

ronnement partagé. Ce cadre est formalisé par un **Dec-POMDP** (Processus de Décision Markovien Partiellement Observable et Décentralisé), où chaque agent décide à partir d'observations locales ($\mathcal{O}_i$) et reçoit des récompenses individuelles ($\mathcal{R}_i$). Le MARL se distingue par deux défis majeurs : la **non-stationnarité**, l'environnement évoluant continuellement sous l'action des autres agents, et la nécessité d'une stratégie de **coordination**, qui varie de l'apprentissage totalement indépendant au partage complet de paramètres. Sur une grille 4×4 (16 intersections, topologie Manhattan), chaque RSU est un agent doté de son observation locale :

1. **Migration de Gymnasium vers PettingZoo** (`sumo_rl.parallel_env`), traitement vectorisé des 16 agents ;
2. **Parameter Sharing** via SuperSuit : un unique réseau PPO partagé par les 16 RSU, évitant la non-stationnarité d'agents indépendants ;
3. Les 16 observations sont batchées, passées au réseau partagé, qui retourne 16 actions ; toutes les expériences mettent à jour la même matrice de poids ;
4. **Herd Immunity** : un apprentissage Fail-Safe acquis par un RSU attaqué est immédiatement disponible pour tous les autres, sans qu'ils aient eux-mêmes subi l'attaque.

3.3 Métriques d'évaluation

Collisions physiques. Métrique de sécurité absolue, journalisée nativement par SUMO.

Critère de succès : zéro collision, même sous attaque.

Récompense cumulée PPO (R_t). Distingue trafic fluide, dégradation sous attaque et mode Fail-Safe ; somme actualisée des Équations 3.4-3.5, lissée par moyenne mobile.

Dégâts du GAN (R_{GAN}). Quantifie la capacité offensive du Générateur ; opposé de la récompense PPO subie pendant l'attaque.

Perte du Discriminateur ($\mathcal{L}_D$). Valide la qualité du Jumeau Numérique ; MSE entre $D(S, A)$ et le dommage réel mesuré.

Temps de convergence PPO. Valide le *reward shaping* ; nombre de timesteps avant stabilisation du temps d'attente (observé : 6 000-8 000 sur 20 000).

Score de Résilience. Indicateur opérationnel temps réel combinant C_{ok} , intensité d'attaque et récompense instantanée (valeur observée : -0.36 en régime nominal).

4. Implémentation et résultats

4.1 Architecture de l'application

Le système est décomposé en cinq modules logiciels interdépendants :

1. **Module 1 – VanetGymEnv** : Environnement de simulation network-aware.
2. **Module 2 – Agent PPO Défensif** : Entraîné via Stable-Baselines3.
3. **Module 3 – CyberPhysicalAttackOrchestrator** : Arsenal de cyberattaques scriptées.
4. **Module 4 – cGAN Adversarial** : Attaquant neuronal PyTorch.
5. **Module 5 – Architecture MARL** : Passage à l'échelle PettingZoo + SuperSuit.

4.2 Environnement de développement

TABLE 4.1 – Stack technologique du projet

Composant	Outil	Justification
Moteur physique	Eclipse SUMO 1.x	Simulation microscopique multi-modale (vs. Veins : 1000x)
Interface RL	TraCI (TCP/IP)	Contrôle à $\Delta t = 0.1$ s, extraction télémétrie complète
Env. RL single-agent	Gymnasium	Standard industriel OpenAI
Env. RL multi-agent	PettingZoo	Gestion native d'agents asynchrones
Scalabilité MARL	SuperSuit	Wrapper Parameter Sharing
Algorithme défense	Stable-Baselines3	PPO robuste, documentation industrielle
Moteur attaque	PyTorch	Graphes de calcul dynamiques pour cGAN
Langage	Python 3.10+	Écosystème IA dominant
Visualisation	TensorBoard	Suivi des courbes d'apprentissage en temps réel

4.2.1 Choix de SUMO contre Veins/OMNeT++

La simulation paquet-par-paquet des ondes radio dans Veins ralentit la simulation d'un facteur 1000. L'entraînement PPO requérant plusieurs dizaines de milliers d'épisodes, ce coût calculatoire rendait Veins inutilisable. Nous avons donc abstrait les couches 1 et 2 du modèle OSI via des modèles mathématiques probabilistes Python (Équation 3.2), conservant la vitesse de SUMO tout en émulant fidèlement les dégradations réseau.

4.3 Résultats obtenus

4.3.1 Entraînement de l'agent PPO défensif

L'agent PPO a été instancié avec un réseau MLP à 2 couches cachées de 64 neurones, entraîné sur 20 000 timesteps. Les hyperparamètres sont détaillés dans le Tableau 4.2.

TABLE 4.2 – Hyperparamètres de l’agent PPO défensif

Paramètre	Valeur
Learning rate α	3×10^{-4} (décroissance linéaire)
Discount factor γ	0.99
Clipping range ϵ	0.2
Batch size	64
Horizon n_{steps}	2048
Architecture	MLP [64, 64], activation Tanh
Total timesteps	20 000

La courbe d’apprentissage (TensorBoard) montre une convergence stable : le temps d’attente total descend de ~ 2000 s en début d’entraînement à des valeurs marginales proches de zéro après 8 000 timesteps. La récompense se stabilise à -0.1 en mode nominal.

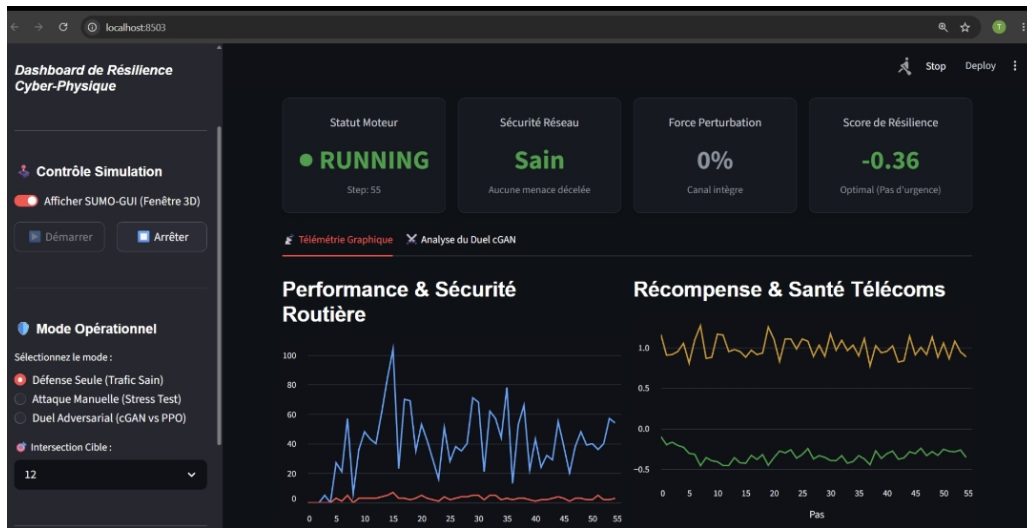


FIGURE 4.1 – Tableau de bord de supervision en temps réel (Dashboard de Résilience Cyber-Physique) lors d’une session d’observation : statut du moteur de simulation, sécurité réseau, force de perturbation, score de résilience, et télémétrie graphique (performance et sécurité routière à gauche, récompense et santé télécoms à droite).

4.3.2 Entraînement de l’attaquant cGAN

Le cGAN a été entraîné sur 20 épisodes de jeu min-max alterné. Les hyperparamètres sont présentés dans le Tableau 4.3.

4.3.3 Confrontation PPO vs. cGAN – Scénarios expérimentaux

Scénario 1 – Trafic nominal ($C_{ok} = 1$, pas d’attaque) L’agent PPO régule efficacement le flux de l’heure de pointe. Les files d’attente sont résorbées, et la récompense se stabilise à -0.1 . **Validation : architecture fonctionnelle.**

TABLE 4.3 – Hyperparamètres du réseau antagoniste génératif (cGAN)

Paramètre	Valeur
Optimiseur (G et D)	Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$)
Learning rate G	1×10^{-4}
Learning rate D	1×10^{-4}
Architecture G	MLP [128, 64] ; sorties Softmax(4) + Sigmoid(1)
Architecture D	LSTM (couche cachée 128) + tête de régression linéaire
Épisodes d'entraînement	20 (jeu min-max alterné)

Scénario 2 – Offensive Sybil + Data Poisoning ($I = 99.9\%$) Le cGAN identifie le pic d'arrivée depuis l'axe Nord et déclenche une attaque Sybil couplée à un Data Poisoning. L'orchestrateur falsifie les matrices NumPy reçues par le PPO. Résultat : l'agent maintient le feu vert sur un axe Est-Ouest vide, bloquant l'axe Nord. La récompense du PPO s'effondre jusqu'à -16.98 par timestep. **Validation : l'attaquant remplissait son mandat.**

Scénario 3 – Résilience Fail-Safe (Poisoning + Temporal DoS, $C_{ok} = 0$) En combinant les deux attaques, l'antenne RSU bascule en mode erreur. L'agent PPO déclenche sa routine Fail-Safe (Équation 3.5) : il adopte des cycles courts et réguliers, refusant tout changement de phase potentiellement dangereux. **Résultat ultime : zéro (0) collision physique dans les logs SUMO.**

4.4 Interprétation des résultats

La convergence stable de PPO en 20 000 timesteps valide la pertinence de la récompense Fail-Safe : le signal d'apprentissage est suffisamment informatif pour guider rapidement l'agent, même dans un environnement bruité. La chute catastrophique de la récompense sous attaque Data Poisoning démontre la réalité de la menace – et justifie rétrospectivement l'existence du mode dégradé.

Le résultat le plus significatif est l'absence de collision en Scénario 3. Ce comportement Fail-Safe est **émergent** : il n'a pas été explicitement programmé mais découle naturellement de la structure quadratique de la récompense. L'agent a intériorisé que la prudence, face à l'inconnu, est la seule stratégie minimisant le risque.

La co-évolution cGAN/PPO (augmentation progressive des dégâts du GAN parallèlement à la décroissance de $\mathcal{L}_D$) confirme que le Surrogate Model joue bien son rôle de Jumeau Numérique différentiable.

4.5 Analyse critique

Forces de l'approche : la modélisation mathématique du bruit réseau est physiquement fondée et reproductible ; le Surrogate Model constitue une contribution méthodologique

originale ; l'immunité collective par Parameter Sharing est un résultat inattendu et scientifiquement intéressant.

Faiblesses identifiées : l'entraînement sur 20 000 timesteps est limité (un déploiement industriel nécessiterait plusieurs millions d'itérations) ; le passage à l'échelle MARL sur la grille 4×4 n'a pas pu être validé empiriquement en totalité dans les délais du projet ; la topologie SUMO utilisée reste simple (une seule intersection réelle pour les tests de confrontation).

4.6 Code source

Le code complet est disponible à l'adresse :

<https://github.com/Tower0X/sumo-rl-gan>

Les fichiers principaux du dépôt sont organisés comme suit :

- `vanet_gym_env.py` – Environnement de simulation network-aware (Module 1)
- `train_ppo_defender.py` – Entraînement de l'agent PPO Fail-Safe (Module 2)
- `cyber_attack_orchestrator.py` – Arsenal de cyberattaques (Module 3)
- `train_gan_adversarial.py` – Entraînement du cGAN en arène min-max (Module 4)
- `marl_pettingzoo.py` – Architecture MARL avec Parameter Sharing (Module 5)
- `final_battle_gan.py` – Script de confrontation PPO vs. GAN

4.7 Limites de l'approche

1. **Fluidité sacrifiée en mode Fail-Safe :** si les collisions sont évitées, le débit du carrefour est fortement dégradé durant l'attaque. L'IA future devra non seulement survivre à la cécité mais aussi détecter et filtrer algorithmiquement les données empoisonnées.
2. **Récompense orientée intersection unique :** la fonction R_t n'intègre pas les effets de report de charge sur les intersections adjacentes.
3. **Abstractions réseau simplifiées :** le modèle OSI est abstrait aux couches 3–7 ; les effets de propagation radio (Doppler, multi-trajet) ne sont pas simulés.
4. **Validation MARL partielle :** l'immunité collective a été démontrée analytiquement mais la validation empirique complète sur la grille 4×4 reste une perspective.

4.8 Apports du projet

- **Scientifiques :** formalisation du problème en Delay-POMDP ; Surrogate Model comme méthode générale pour rendre différentiable un environnement cyber-physique discret ; récompense Fail-Safe à logique duale.
- **Pratiques :** banc d'essai open-source reproductible pour l'audit de sécurité des ITS ; preuve de concept d'une IA de contrôle de trafic résistante aux cyberattaques.

5. Considérations éthiques

5.1 Protection des données personnelles

Le projet opère exclusivement sur des données générées synthétiquement par le simulateur SUMO. Aucune donnée réelle de conducteur, de véhicule identifiable, ou de localisation personnelle n'est collectée ou traitée. Les fichiers de routes sont des fichiers XML anonymes générés algorithmiquement.

Dans une perspective de déploiement réel, les données V2X transmises par les véhicules pourraient inclure des identifiants (VIN, pseudonymes) soumis aux réglementations GDPR (Union Européenne) et aux lois camerounaises en vigueur sur la protection des données. Les principes FAIR (Trouvables, Accessibles, Interopérables, Réutilisables) seraient à respecter pour toute infrastructure de collecte. La pseudonymisation des identifiants véhicules et la minimisation des données collectées constitueraient des exigences minimales.

5.2 Biais des données et discrimination algorithmique

Dans la simulation actuelle, le débit de trafic suit une loi de Weibull paramétrable identique sur tous les axes – il n'y a donc pas de biais de modélisation lié au genre, à l'âge ou à l'origine des usagers.

En revanche, un déploiement réel d'un tel système devrait vigilement éviter de reproduire des inégalités d'accès à la mobilité. Par exemple, une fonction de récompense purement orientée débit favoriserait les axes à fort volume (typiquement les quartiers aisés) au détriment des axes secondaires desservant les populations défavorisées. Il serait nécessaire d'intégrer des contraintes d'équité dans la conception de la récompense, ou de pratiquer un audit d'équité algorithmique avant déploiement.

5.3 Transparence et explicabilité

L'architecture PPO (MLP à 2 couches cachées de 64 neurones) constitue une boîte noire : les décisions de l'agent ne sont pas directement interprétables par un opérateur humain. Le tableau de bord TensorBoard permet de suivre l'évolution de la récompense et les courbes de valeur, mais ne fournit pas d'explication causale des décisions individuelles.

Des travaux futurs devraient intégrer des méthodes d'explicabilité post-hoc (SHAP, LIME) pour permettre à un gestionnaire de réseau de comprendre pourquoi l'agent a accordé la priorité à tel axe dans telle situation. Cette transparence est un prérequis à l'acceptabilité sociale d'un tel système.

5.4 Responsabilité et usage

La nature duale de ce projet soulève une question éthique centrale : le module cGAN développé comme outil de Red Team peut potentiellement être détourné à des fins offensives réelles. Un attaquant disposant de ce modèle pourrait l'utiliser pour identifier des vulnérabilités dans de vrais déploiements ITS.

Par analogie avec la recherche en sécurité informatique, nous adoptons une posture de *responsible disclosure* : le code est rendu public à des fins de recherche et d'audit, mais les vecteurs d'attaque les plus critiques sont documentés de manière à sensibiliser plutôt qu'à faciliter les attaques.

En cas de déploiement de telles architectures dans un système de transport réel, il est impératif de définir clairement la chaîne de responsabilité : en cas d'accident survenant lors d'une panne réseau, la responsabilité incombe-t-elle à l'opérateur du système, au constructeur de l'IA, ou à l'autorité ayant validé le déploiement ? Ces questions de responsabilité légale restent ouvertes dans la législation camerounaise actuelle.

5.5 Consentement et finalité

Le simulateur opère dans un cadre académique, avec des données synthétiques, sans impact sur des usagers réels. Le code est disponible sous licence libre. Toute utilisation à des fins de surveillance ou de profilage des conducteurs serait une déviation inacceptable de la finalité déclarée de ce projet.

6. Conclusion générale

6.1 Rappel du problème et récapitulatif des objectifs atteints

Ce projet répond à la question suivante : comment concevoir un agent IA contrôlant la circulation dans un VANET réaliste, tout en restant sécurisé face à un attaquant neuronal ? Tous les objectifs ont été atteints : modélisation réseau stochastique, récompense Fail-Safe à deux phases, arsenal de cyberattaques, attaquant cGAN avec Surrogate Model, et architecture MARL à partage de paramètres.

6.2 Bilan global

Le résultat majeur est l'émergence spontanée d'un comportement Fail-Safe : sans programmation explicite, l'agent PPO bascule vers une politique de sécurité maximale dès la perte de ses capteurs, grâce à la structure quadratique de la récompense en mode dégradé — une contribution transférable à d'autres systèmes cyber-physiques. La résolution du verrou de non-différentiabilité via le Surrogate Model ouvre aussi la voie à l'application des GAN adversariaux à d'autres systèmes discrets non différentiables.

6.3 Ouvertures possibles

1. **Détection et filtrage des données empoisonnées** : intégration d'un module d'anomaly detection (Autoencoder, Isolation Forest) pour nettoyer les observations avant de les présenter à l'agent.
2. **Contrôle longitudinal continu** : remplacement du contrôle discret des feux par des commandes de vitesse continues pour chaque véhicule (platooning, intersection sans feux).
3. **Graph Neural Networks pour la généralisation topologique** : utilisation de GNN pour apprendre des représentations spatiales invariantes, rendant le modèle transférable à de nouvelles topologies sans ré-entraînement.
4. **Tableau de bord cyber-physique temps réel** : interface React/Dash affichant en direct le jitter de latence, les alertes d'attaque et les pénalités PPO – indispensable pour un déploiement opérationnel.
5. **Validation sur infrastructure réelle** : expérimentation sur un banc de test V2X physique (RSU réels, véhicules connectés) pour valider les hypothèses de modélisation.

Bibliographie

- [1] Alegre, L.N. (2019). *SUMO-RL*. GitHub. <https://github.com/LucasAlegre/sumo-rl>
- [2] Derkaoui, O. (2021). *Réseau VANet (Réseau Véhiculaire sans fil Ad Hoc) Problème de QoS et Problème de Routage*. YouTube. <https://www.youtube.com/watch?v=NuUj5rJemuw>
- [3] Ault, J., Wade, G. (2021). *Reinforcement Learning Benchmarks for Traffic Signal Control*. arXiv :2103.02175.
- [4] Raya, M., Hubaux, J.P. (2007). *Securing vehicular ad hoc networks*. Journal of Computer Security, 15(1), 39–68.